

GoGov Integration

Status: mock only, and deliberately so

GoGov does not publish a public API specification. This project therefore defines the *shape* an integration would take and ships an explicitly labeled mock. No endpoint URL, authentication scheme, or payload format in this codebase is claimed to match a real GoGov service, because that information is not publicly available.

Inventing plausible-looking endpoints would be worse than having none: it produces code that looks finished, fails in unpredictable ways, and misleads whoever picks it up next.

The abstraction

backend/app/integrations/gogov.py defines:

```
class GoGovService(ABC):
    async def search_faqs(query, *, limit=5)
    async def create_request(*, department, summary, deta
    async def get_request_status(request_id)
    async def generate_service_link(*, department, servic
```

Two implementations:

	MockGoGovService	LiveGoGovService
GOGOV_MODE	mock (default)	live

Behavior	Labeled fake responses	Raises <code>NotImplementedError</code> with instructions
UI badge	MOCK GOV SERVICE	LIVE GOV SERVICE

Every mock response carries `"mode": "mock", "is_live": false`, and a disclaimer string. Check `GET /api/gogov/status` to see which is active.

Why `search_faqs` returns nothing

The mock returns an empty result set rather than invented FAQ text. Fabricated municipal answers entering the RAG pipeline is precisely the failure this system is built to prevent — and a mock that returns fake answers would be indistinguishable from a working integration during a demo.

What the Village must obtain from GoGov

Before any implementation work is possible:

1. **API documentation** — base URL, endpoints, request/response schemas, error codes
2. **Authentication method** — API key, OAuth 2.0 client credentials, or mTLS
3. **Credentials** — issued for the Village's account, with separate sandbox and production sets
4. **A sandbox environment** — testing against production would create real service requests for real residents
5. **Rate limits and quotas**
6. **A data processing agreement** — resident contact details would flow to GoGov
7. **Field mapping** — how GoGov's service categories align with this system's nine departments

8. **Webhook support** (if available) — for status changes, avoiding polling

Most of this is a contract and account-management conversation, not an engineering task. GoGov's API access is typically tied to the municipality's existing service agreement.

Implementation, once documentation exists

1. Implement the four methods in `LiveGoGovService`
2. Add `GOGOV_BASE_URL` and `GOGOV_API_KEY` to `.env` (the keys already exist)
3. Map GoGov categories to department IDs in `config/departments.yaml`
4. Set `GOGOV_MODE=live`
5. Verify `GET /api/gogov/status` reports `LIVE GOV SERVICE`

No other file changes — the abstraction is the same seam used for LLM, STT, and TTS providers.

Requirements for the live implementation

- **Timeouts and retries** with exponential backoff. A GoGov outage must degrade to "I can take your request another way", never a hung call.
 - **Circuit breaker.** Repeated failures should stop attempts rather than making every caller wait for a timeout.
 - **No silent failure.** If a request was not created, the resident must be told.
 - **Contact data minimization.** Collect only what GoGov requires, store none of it locally, and log none of it.
 - **Idempotency.** A retried call must not create duplicate service requests.
-
-

Where it fits in the conversation

Once live, the natural flow is:

```
Resident reports an issue ("there's a pothole on Stewart")
↓
Routed to Public Works, high confidence
↓
Assistant offers to file a service request
↓
Resident confirms and provides a location
↓
create_request() → GoGov
↓
Assistant reads back the confirmation number
```

This is not implemented. It requires deciding how much contact information the assistant may collect and store, which is a privacy question for the Village (see [SECURITY ROADMAP.md](#)), not a technical default this prototype should assume.

If GoGov has no usable API

Two workable alternatives:

Deep links. `generate_service_link()` already returns a URL. The assistant can say "I can text or email you a link to the request form" and hand off to the existing GoGov web flow. This requires no API at all and is often the fastest path to value.

Email or webhook bridge. Many municipal platforms accept structured email into a service queue. `create_request()` could format and send that instead, with the same interface.

Both keep the abstraction intact — only `LiveGoGovService` changes.